Blueprint - Complete Technical PRD (MVP)

1. Project Overview

Portfolio-grade visual form builder. Users authenticate, create blueprints, build forms in a vertical visual editor, publish immutable forms, collect responses and review submissions.

2. Tech Stack

Frontend: Next.js, Tailwind, React Flow, Better Auth client. Backend: Express, Drizzle ORM, PostgreSQL. API-first architecture. Deployment: Vercel(frontend)+backend service.

3. MVP Features

Authentication (email/password + Google), Dashboard, Builder, Draft/Publish lifecycle, Public one-question-at-a-time form, Response table, Individual response viewer, Duplicate blueprint.

4. Non-MVP

Conditional logic, analytics, collaboration, file uploads, themes, editing published forms.

5. User Flow

Login → Dashboard → Create Blueprint → Builder → Publish → Public URL → Submit Response → Owner views responses.

6. Builder

React Flow used as vertical canvas only. Questions rendered with fixed gap. Database stores only order_index; node positions are generated client-side.

7. Database Schema

users

column	type
id	uuid pk
name	text
email	text unique
created_at	timestamp

blueprints

column	type
id	uuid
owner_id	fk users
title	text
description	text
status	enum(draft,published,closed,archived)

public_id	text unique
created_at	timestamp

questions

column	type
id	uuid
blueprint_id	fk
title	text
description	text nullable
type	enum
required	bool
order_index	int

question_options

column	type
id	uuid
question_id	fk
label	text
order_index	int

responses

column	type
id	uuid
blueprint_id	fk
submitted_at	timestamp
completion_ms	int nullable
ip_hash	text nullable
user_agent	text nullable

answers

column	type
id	uuid
response_id	fk
question_id	fk
option_id	fk nullable
value	text nullable

8. ER Relationships

User 1-N Blueprints. Blueprint 1-N Questions. Question 1-N Options. Blueprint 1-N Responses. Response 1-N Answers. Question 1-N Answers.

9. REST API

POST/GET/PATCH/DELETE /blueprints, POST/PATCH/DELETE questions, POST/PATCH/DELETE options, GET public blueprint, POST responses, GET responses, GET response by id.

10. Validation

Validate on frontend and backend using shared Zod schemas. Published blueprints cannot be modified.

11. Frontend Architecture

Pages: auth, dashboard, builder, responses, public form. Components: Sidebar, Canvas, QuestionNode, PropertiesPanel, Toolbar. Zustand for builder state.

12. Backend Architecture

Routes → Controllers → Services → Repositories → PostgreSQL. Use transactions for create/publish/submit operations.

13. Folder Structure

apps/web, apps/api, packages/shared. Shared types and validation.

14. Indexes

Unique: users.email, blueprints.public_id. Index: blueprint_id on questions/responses, response_id and question_id on answers.

15. Development Phases

1 Auth. 2 Dashboard. 3 Builder. 4 Publish. 5 Public form. 6 Responses. 7 Polish/UI.